



UNIVERSIDAD TECNOLÓGICA NACIONAL (UTN)
TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

MATERIA: **PROGRAMACIÓN I**

TÍTULO DEL PROYECTO: TRABAJO PRÁCTICO INTEGRADOR II: *Gestión de datos de países en Python: filtros, ordenamientos y estadísticas*

ALUMNOS:

- BARRAGAN, Lucas
- MOREYRA, Lorenzo

FECHA DE ENTREGA: 15/06/26

ÍNDICE

Marco Teórico	2
Decisiones Técnicas y Arquitectura	6
Dificultades y Conclusiones.....	8
Bibliografía / Webgrafía.....	10
Links	11

Marco Teórico

En el desarrollo de software y la programación, existen diversas estructuras y herramientas fundamentales que permiten organizar, procesar y almacenar información de manera eficiente. Entre ellas se encuentran los arrays, diccionarios, funciones, estructuras condicionales, algoritmos de ordenamiento y archivos CSV. Estos conceptos constituyen la base para la construcción de programas capaces de manipular datos, tomar decisiones y gestionar información de forma organizada.

Arrays o Listas

Un array es una estructura de datos que permite almacenar múltiples elementos del mismo tipo, organizados de manera ordenada. Cada elemento se encuentra en una posición específica identificada mediante un índice, el cual comienza desde cero.

Una de las principales características de los arrays es que todos los elementos almacenados deben ser homogéneos, es decir, pertenecer al mismo tipo de dato, como números, cadenas de texto o valores booleanos. Esto facilita el acceso y procesamiento de la información.

Puede compararse con una fila de casilleros numerados, donde cada espacio contiene un dato y puede ser accedido directamente a través de su número de posición. Los arrays son ampliamente utilizados para almacenar colecciones de datos que posteriormente pueden ser recorridas, analizadas, ordenadas o modificadas mediante diferentes algoritmos y estructuras de control.

Diccionarios

Los diccionarios son estructuras de datos compuestas por pares clave-valor (key-value). A diferencia de los arrays, los elementos no se identifican mediante posiciones numéricas, sino mediante claves únicas que permiten acceder directamente a la información asociada.

Esta estructura resulta especialmente útil cuando se necesita representar datos relacionados entre sí, como por ejemplo un nombre y una edad, un producto y su precio o un identificador y su descripción. Los diccionarios permiten organizar la información de manera flexible y eficiente, facilitando la búsqueda y recuperación de datos específicos.

Mientras que los arrays son adecuados para almacenar conjuntos ordenados de elementos similares, los diccionarios resultan más apropiados para representar información estructurada donde cada dato posee un identificador propio.

Funciones

Una función es un bloque de código reutilizable diseñado para realizar una tarea específica. Su principal objetivo es evitar la repetición de instrucciones y facilitar la organización del programa.

Las funciones permiten dividir problemas complejos en partes más pequeñas y manejables, favoreciendo la modularidad del código. Además, contribuyen a mejorar la claridad y el mantenimiento de los programas, ya que cualquier modificación o corrección puede realizarse en un único lugar.

Entre sus principales ventajas se destacan:

- Reutilización del código en diferentes partes del programa.
- Modularidad, al dividir aplicaciones complejas en componentes más simples.
- Mayor claridad y legibilidad.
- Facilidad de mantenimiento y corrección de errores.
- Reducción de la redundancia mediante la aplicación del principio DRY (Don't Repeat Yourself).

Las funciones suelen trabajar conjuntamente con arrays, diccionarios y archivos CSV, ya que permiten encapsular procesos de lectura, búsqueda, ordenamiento o análisis de datos.

Estructuras Condicionales

Las estructuras condicionales son mecanismos de control que permiten a un programa tomar decisiones en función del cumplimiento o no de determinadas condiciones. Constituyen la base del pensamiento lógico en programación y permiten desarrollar aplicaciones dinámicas capaces de responder a diferentes situaciones.

Su funcionamiento puede resumirse de la siguiente manera:

- Si se cumple una condición, se ejecuta una acción.
- Si no se cumple, se ejecuta una acción alternativa o no se realiza ninguna operación.

Condición simple (if)

La estructura **if** ejecuta un bloque de código únicamente cuando una condición se evalúa como verdadera. Si la condición es falsa, el bloque se omite.

Condición compuesto (if-else)

La estructura **if-else** incorpora una alternativa para los casos en que la condición resulte falsa. De esta manera, siempre se ejecutará uno de los dos bloques definidos.

Condición múltiple (if-elif-else)

Cuando es necesario evaluar varias condiciones diferentes, se utiliza la estructura **if-elif-else**. El programa analiza cada condición en orden y ejecuta el bloque correspondiente a la primera condición verdadera. Si ninguna se cumple, puede ejecutarse el bloque **else**.

Condicionales anidados

Los condicionales anidados consisten en colocar una estructura condicional dentro de otra. Esto permite construir procesos de decisión más complejos y establecer relaciones de dependencia entre distintas condiciones.

Operadores lógicos

Las condiciones pueden combinarse mediante operadores lógicos:

- **and**: devuelve verdadero únicamente cuando todas las condiciones son verdaderas.
- **or**: devuelve verdadero cuando al menos una de las condiciones es verdadera.
- **not**: invierte el resultado lógico de una condición.

Las estructuras condicionales suelen utilizarse junto con funciones, arrays y diccionarios para analizar datos y ejecutar diferentes acciones según los resultados obtenidos.

Algoritmos de Ordenamiento

Los algoritmos de ordenamiento son procedimientos utilizados para reorganizar elementos de una colección de datos siguiendo un criterio determinado, como orden ascendente o descendente.

Uno de los métodos más conocidos consiste en comparar pares de elementos adyacentes e intercambiarlos cuando se encuentran en un orden incorrecto. Este proceso se repite sucesivamente hasta que todos los elementos quedan correctamente organizados.

Los algoritmos de ordenamiento resultan fundamentales para optimizar la búsqueda y el análisis de información almacenada en arrays o listas, mejorando la eficiencia en el procesamiento de datos.

Archivos CSV

Un archivo CSV (Comma Separated Values o Valores Separados por Comas) es un archivo de texto plano diseñado para almacenar información tabular. Aunque visualmente puede parecer un simple archivo de texto, sigue una estructura específica que organiza los datos en filas y columnas.

Los archivos CSV son ampliamente utilizados en el desarrollo de software y la ciencia de datos debido a diversas ventajas:

- **Interoperabilidad:** permiten intercambiar información entre diferentes sistemas y lenguajes de programación.
- **Persistencia:** posibilitan almacenar datos de manera permanente en dispositivos de almacenamiento.
- **Ligereza:** ocupan menos espacio que otros formatos más complejos, como los archivos de hojas de cálculo.
- **Entrada de datos:** facilitan la carga masiva de registros, configuraciones o información histórica.

Lectura de archivos CSV en Python

Python dispone del módulo **csv**, incluido en su biblioteca estándar, que proporciona herramientas específicas para leer y procesar este tipo de archivos.

Para utilizarlo es necesario importar previamente el módulo mediante la instrucción:
`import csv`

A través de este módulo, los datos pueden transformarse en estructuras manipulables, como listas o diccionarios, permitiendo su posterior procesamiento mediante funciones, estructuras condicionales y algoritmos de ordenamiento.

Uso del parámetro `encoding="utf-8"`

Al abrir un archivo CSV mediante la función `open()`, es recomendable especificar el parámetro `encoding="utf-8"`.

El encoding determina la forma en que Python interpreta los bytes almacenados en el archivo y los convierte en caracteres legibles. UTF-8 es el estándar más utilizado a nivel mundial y garantiza la correcta visualización de caracteres especiales, como tildes, eñes y símbolos propios de distintos idiomas.

La utilización de este parámetro evita errores de lectura y asegura la compatibilidad de los datos entre diferentes sistemas operativos y aplicaciones.

A modo concluyente, se puede decir que los conceptos desarrollados se encuentran estrechamente relacionados dentro del proceso de programación. Los archivos CSV permiten almacenar y recuperar información, la cual puede cargarse en arrays o diccionarios para ser manipulada dentro del programa. Posteriormente, las funciones facilitan la organización del código encargado de procesar esos datos, mientras que las estructuras condicionales permiten tomar decisiones en función de determinadas condiciones. Finalmente, los algoritmos de ordenamiento posibilitan reorganizar la información obtenida para presentarla de forma más clara y eficiente.

En conjunto, estas herramientas constituyen elementos esenciales para el desarrollo de programas capaces de gestionar información de manera organizada, reutilizable y eficiente.

Decisiones Técnicas y Arquitectura

La aplicación fue desarrollada siguiendo una arquitectura modular basada en funciones. Cada operación principal se encuentra centrada en una función independiente, lo que mejora la organización, la reutilización y el mantenimiento del código. Los datos de los países se almacenan en una lista de diccionarios, permitiendo representar cada país mediante atributos como nombre, población, superficie y continente.

Carga inicial de datos
desde CSV

Los datos se leen desde un archivo CSV y se transforman en diccionarios de Python, lo que permite una representación estructurada y fácil de manipular.



Uso de diccionarios

Cada país se almacena como un diccionario, ya que este formato permite acceder a los atributos mediante claves descriptivas (nombre, población, etc.), mejorando la legibilidad del código.



Modularización mediante
funciones

Las operaciones principales (agregar, actualizar, buscar y filtrar) se implementaron como funciones independientes para favorecer la reutilización y el mantenimiento.



Validación de entradas

Se incorporaron validaciones para evitar datos vacíos o valores numéricos inválidos, mejorando la robustez del sistema.



Menú interactivo

La aplicación utiliza un bucle principal que muestra un menú y dirige la ejecución hacia la función correspondiente según la opción seleccionada por el usuario.

Diagrama de flujo:

MENU PRINCIPAL



SELECCIÓN DE OPCION



AGREGAR PAIS
ACTUALIZAR PAIS

BUSCAR PAIS

FILTRAR PAISES

ORDENAR PAISES

MOSTRAR ESTADISTICAS



VOLVER AL MENU



FINALIZAR PROGRAMA

MENU PRINCIPAL Y SELECCIÓN DE OPCION

```
=== GESTIÓN DE PAÍSES ===  
1. Agregar país  
2. Actualizar país  
3. Buscar país  
4. Filtrar países  
5. Ordenar países  
6. Mostrar estadísticas  
0. Salir  
Seleccione una opción: █
```

FUNCIONES

Seleccione una opción: 4

```
=== FILTROS ===  
1. Filtrar por continente  
2. Filtrar por población  
3. Filtrar por superficie  
Seleccione una opción: 2  
Ingrese la población mínima: 10000  
Ingrese la población máxima: 4444444  
{'nombre': 'Uruguay', 'poblacion': 3426260, 'superficie': 176215, 'continente': 'America'}
```

Seleccione una opción: 3
Ingrese el nombre del país: 12
No se encontraron países.

VOLVER AL MENU

```
=== GESTIÓN DE PAÍSES ===  
1. Agregar país  
2. Actualizar país  
3. Buscar país  
4. Filtrar países  
5. Ordenar países  
6. Mostrar estadísticas  
0. Salir
```

SALIR

```
=== GESTIÓN DE PAÍSES ===  
1. Agregar país  
2. Actualizar país  
3. Buscar país  
4. Filtrar países  
5. Ordenar países  
6. Mostrar estadísticas  
0. Salir  
Seleccione una opción: 0  
Programa finalizado.
```

Dificultades y Conclusiones

DESAFIOS TECNICAS

Durante el desarrollo del trabajo las dificultades que se fueron presentado estaban relacionadas con el manejo de los datos de los países. Una de las más importantes fue lograr que las funciones de búsqueda y actualización encontraran correctamente el país solicitado por el usuario, especialmente cuando había que validar los datos ingresados.

También fue un desafío el correcto funcionamiento del programa al realizar los filtros y ordenamientos, ya que tuvimos que pensar la lógica necesaria para recorrer la lista de países y mostrar únicamente la información que cumpliera con determinadas condiciones.

Otra complicación fue la generación de estadísticas, debido a que fue necesario analizar los datos almacenados y realizar distintos cálculos para obtener resultados correctos.

Fue un concepto muy útil el uso de Try/Except para la validación de los datos ingresados por el usuario para evitar errores durante la ejecución del programa además de ello utilizamos estructuras condicionales.

Por último, a medida que el programa crecía, se volvió más difícil mantener el código organizado. Para solucionar este problema dividimos el sistema en varias funciones, lo que nos permitió trabajar de una forma más ordenada y facilitar la lectura y el mantenimiento del programa.

DESAFIOS COLABORATIVOS

Otro de los desafíos que se nos presentó en la realización del TPI fue la del trabajo colaborativo. Al tratarse de compañeros que no se conocían previamente, fue necesario establecer una buena comunicación y organización para distribuir las tareas y mantener el proyecto integrado.

CONCLUSIONES

La realización de este Trabajo Practico Integrador nos permitió poner en practica todos los conocimientos de la materia aprendidos a lo largo del cuatrimestre. A partir del desarrollo del mismo pudimos entender mejor el uso de las herramientas que nos brindaron desde el equipo docente como son: funciones, listas, diccionarios, estructuras de control, manejo de archivos y tratamiento de excepciones, la importancia de la organización del código y del diseño modular para mantener su funcionabilidad mantenimiento y comprensión.

A su vez, también, el trabajo colaborativo nos invita a desarrollar habilidades de comunicación, compañerismo y trabajo en equipo, aspectos que consideramos clave en el mundo IT. Esta experiencia nos ayudó a mejorar la organización, la distribución de responsabilidades y la capacidad de resolver problemas en conjunto.

Bibliografía / Webgrafía

Enlaces a la documentación oficial o recursos utilizados como apoyo.

- Apuntes de Catedra brindados dentro del Campus Virtual.
- Herramientas de IA: ChatGPT y Claude.ai

Links

al repositorio del proyecto y link del video explicativo solicitado más abajo.

Video de Youtube: <https://youtu.be/IsMc0Z0VrW8>

Repositorio de Github: <https://github.com/LucasBarragan/UTN-TPI-PROGRAMACION>